This WACV paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

# Point-JEPA: A Joint Embedding Predictive Architecture for Self-Supervised Learning on Point Cloud

Ayumu Saito Prachi Kudeshia
Jiju Poovvancheri
Graphics and Spatial Computing Lab, Saint Mary's University
Halifax, Canada
ayumu.saito@smu.ca, prachi.kudeshia@smu.ca, jiju.poovvancheri@smu.ca

## Abstract

*Recent advancements in self-supervised learning in the point cloud domain have demonstrated significant potential. However, these methods often suffer from drawbacks such as lengthy pre-training time, the necessity of reconstruction in the input space, and the necessity of additional modalities. In order to address these issues, we introduce Point-JEPA, a joint embedding predictive architecture designed specifically for point cloud data. To this end, we introduce a sequencer that orders point cloud patch embeddings to efficiently compute and utilize their proximity based on their indices during target and context selection. The sequencer also allows shared computations of the patch embeddings' proximity between context and target selection, further improving the efficiency. Experimentally, our method demonstrates state-of-the-art performance while avoiding the reconstruction in the input space or additional modality. In particular, Point-JEPA attains a classification accuracy of* $\mathbf{93.7}_{\pm 0.2}$ *% for linear SVM on ModelNet40 surpassing all other self-supervised models. Moreover, Point-JEPA also establishes new state-of-the-art performance levels across all four few-shot learning evaluation frameworks. The code is available at* https://github.com/Ayumu-J-S/Point-JEPA

## 1. Introduction

The growing accessibility of affordable consumer-grade 3D sensors has led to the widespread adoption of point clouds as a preferred data representation for capturing real-world environments. However, the existing point cloud understanding approaches [14] mostly rely on supervised training which requires time-consuming and labor-intensive manual annotations to semantically understand 3D environments. On the other hand, self-supervised learning (SSL) is an evolving paradigm that allows the model to learn a meaningful representation from unlabeled data. The success of self-supervised learning in advancing natural language processing and 2D computer vision has motivated its application in the point cloud domain for achieving state-of-the-art results on downstream tasks [17]. However, our initial investigation found that they require a significant amount of pre-training time as shown in Fig. 1. The slow pre-training process can pose constraints in scaling to a larger dataset or complex and deeper models, hindering the key advantage of self-supervised learning; its capacity to learn a strong representation from a vast amount of data.

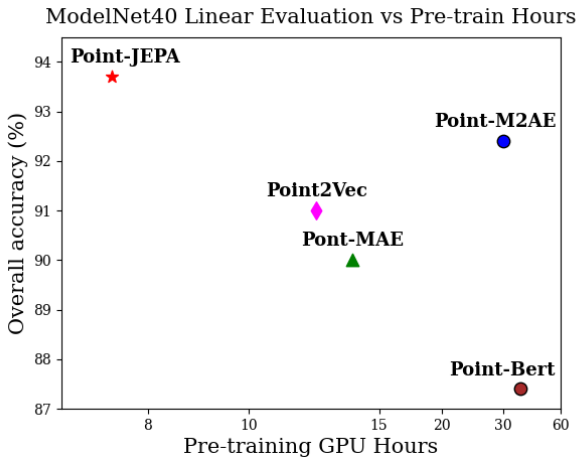



Figure 1. **ModelNet40 Linear Evaluation.** Pre-training time on NVIDIA RTX A5500 and overall accuracy with SVM linear classifier on ModelNet40 [36]. We compare PointJEPA with previous methods utilizing standard Transformer architecture.

The successful implementations of Joint-Embedding Predictive Architecture (JEPA) [18] for pre-training a model [2,3] show JEPA's ability to learn strong semantic representations without the need for fine-tuning. The idea behind JEPA is to learn a representation by predicting the embed-

ding of the input signal, called *target*, from another compatible input signal, called *context*, with the help of a predictor network. This allows learning in the representation space instead of the input space, leading to efficient learning. Inspired by I-JEPA [2], we aim to apply Joint-Embedding Predictive Architecture in the point cloud domain, which introduces a promising direction for self-supervised learning in the point cloud understanding. However, unlike images, unordered point clouds pose a unique challenge to applying JEPA due to their inherently permutation-invariant nature. The unordered nature of the point cloud data makes the context and target selection of the data difficult and inefficient, especially if we aim to select spatially contiguous patches similar to I-JEPA [2]. Therefore, we introduce Point-JEPA to overcome this challenge, while utilizing the full potential of Joint-Embedding Predictive Architecture for computational efficiency. Point-JEPA utilizes an efficient greedy sequencer to assist the model in selecting patch embeddings that are spatially adjacent. Our empirical studies indicate that Point-JEPA efficiently learns semantic representations from point cloud data with faster pre-training times compared to alternative state-of-the-art methods. The specific contributions of this work are as follows.

- We present a Joint-Embedding Predictive Architecture, called Point-JEPA, for point cloud self-supervised learning. Point-JEPA efficiently learns a strong representation from point cloud data without reconstruction in the input space or additional modality.
- We propose a point cloud patch embedding ordering method for Joint-Embedding Predictive Architecture, utilizing a greedy algorithm based on spatial proximity.

## 2. Related Work

Recent advancements in self-supervised learning in 2D computer vision [2, 5, 8, 13, 15, 17, 23, 35] and natural language processing [4, 9, 28, 29] have inspired its application to point cloud processing. In this section, we review existing self-supervised learning methods in the point cloud domain and explore the concept of the Joint Embedding Predictive Architecture.

### 2.1. Generative Learning

Generative models learn representations by reconstructing the input signal within the same input space, capturing its underlying structure and features. For example, based on a popular NLP model Bert [9], Point-Bert [37] introduces generative pretraining to the point cloud using a discrete variational autoencoder to transform the point cloud into discrete point tokens. However, this model heavily relies on data augmentation and suffers from the early leakage of location information, which makes pre-training steps relatively complicated and computationally expensive. To overcome this issue, Point-MAE [25] presents a lightweight, flexible, and computationally efficient solution by bypassing the tokenization and reconstructing the masked point cloud patches. On the other hand, PointGPT [7] introduces an auto-regressive learning paradigm in the point cloud domain. Such generative pre-training in the point cloud domain learns a robust representation; however, it suffers from computational inefficiency due to the reconstruction of the data in the input space.

### 2.2. Joint Embedding Architecture

Joint Embedding Architectures map the input data into a shared latent space that contains similar embeddings for semantically similar instances. These networks utilize regularization strategies such as contrastive learning and self-distillation to learn meaningful representations. Contrastive learning generates embeddings that are close for positive pairs and distant for negative pairs. For example, Du *et al.* [10] introduces a contrastive learning approach that treats different parts of the same object as negative and positive examples. Unlike contrastive learning, a self-distillation network employs two identical networks with distinct parameters, commonly known as the teacher and student, where the teacher guides the student by providing its predictions as targets. For example, in Point2Vec [38], the teacher receives the patches of point clouds while the student receives a subset of these patches. Further, a shallow Transformer learns meaningful and robust representation from the masked positional information and the contextualized embedding from the partial-view input. In self-distillation networks, no reconstruction in the input space results in faster training than in generative models. However, as shown in Fig. 1, it requires longer training to learn a meaningful representation. On the other hand, contrastive learning excels in performance; however, its effectiveness highly depends on the careful selection of positive and negative samples as well as the data augmentation techniques to ensure transferable representations for downstream tasks [17].

### 2.3. Joint Embedding Predictive Architecture (JEPA)

A self-supervised learning architecture JEPA [24] learns representation using a predictor network that predicts one set of encoded signal $y$ based on another set of encoded signal $x$, along with a conditional variable $z$ that controls the prediction. In the predictor network, encoders initially process both the target and the context signals to represent them in embedding space. Conceptually JEPA has a large similarity to generative models which are designed to reconstruct masked part of the input. However, instead of directly operating on the input space, JEPA makes predic-

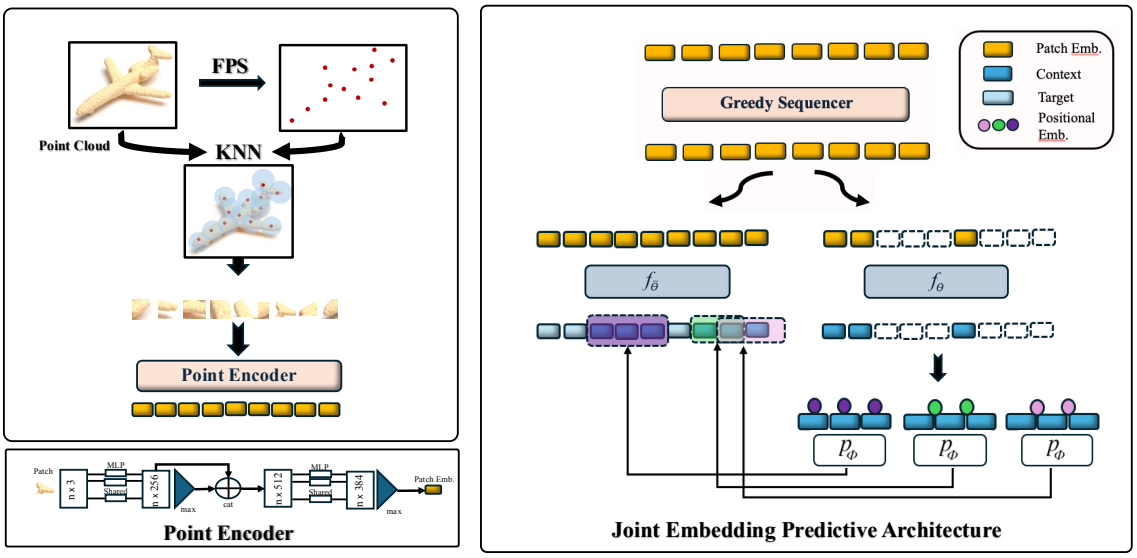



Figure 2. **Schematic renderings illustrating the process of creating embeddings. (Top left), point encoder (bottom left) and Point-JEPA (right).** Point cloud patches are generated using furthest point sampling (FPS) [11] and $k$-nearest neighbor (KNN) methods, a mini PointNet (Point Encoder) is used to generate patch embeddings which are subsequently fed to the JEPA architecture. We use standard Transformer [34] architecture for context ($f_\theta$) and target ($f_{\bar{\theta}}$) encoders as well as predictor ($p_\phi$).

tions in the embedding space. This allows the elimination of unnecessary input details to focus on learning meaningful representations. As a result, the model can abstract and represent the data more efficiently. Closely related to our work, the specific application of the architecture in the image domain can be seen in I-JEPA [2]. In this work, the context signal is created by selecting a block of patches while the target signals are created by sampling the rest of unselected patches. Experiments show faster convergence of I-JEPA to learn highly semantic representation. Therefore, to ensure faster pretraining in self-supervised learning for point cloud understanding, we aim to apply JEPA on point cloud data.

## 3. Point-JEPA Architecture

In this section, we describe our JEPA architecture for pretraining in the point cloud domain. Our goal is to adapt JEPA [18] for use with point cloud data while evaluating its performance and implementation efficiency. The overall framework, as shown in Fig. 2, first converts the point cloud to a set of patch embeddings, then a greedy sequencer arranges them in sequence based on their spatial proximity to each other, and Joint-Embedding Predictive Architecture is applied to the ordered patch embeddings. We utilize a mini PointNet [26] architecture for encoding the grouped points and standard Transformer [34] architecture for the context and target encoder as well as the predictor. It is important to note that our JEPA architecture operates on embeddings instead of patches in order to share the point encoder network between context and target encoder for efficiency similar to Point2Vec [38].

### 3.1. Point Cloud Patch Embedding

Building on previous studies that utilize the standard Transformer architecture for point cloud objects [7, 25, 38], we adopt a process that embeds groups of points into patch embeddings. Given a point cloud object, $P \subset \mathbb{R}^3$ consisting of $n$ points, $c$ center points are first sampled using the farthest point sampling [11]. Then we employ the $k$-nearest neighbors algorithm to identify and select the $k$ closest points surrounding each of the $c$ designated center points. These point patches are then normalized by subtracting the center point coordinates from the coordinates of the points in the patches. This allows the separation between local structural information and the positional information of the patches. In order to embed the local point patches, we utilize a mini PointNet [26] architecture. This ensures that the patch embedding remains invariant to any permutations of data feeding order of points within the patch. Specifically, this PointNet contains two sets of a shared multi-layer perceptron (MLP) and a max-pooling layer as shown in Fig. 2. First, a shared MLP maps each point into a feature vector. Then, we apply max-pooling to these vectors and concatenate the result back to the original feature vector. Subsequently, a shared MLP processes these concatenated vectors, followed by a max-pooling operation to generate a set of patch embeddings $T$ of $P$.

**Algorithm 1:** Greedy sequencer strategy

**Input :** Set of patch emb., $T = \{t_1, t_2, .., t_r\}$
**Output:** Set of spatially contiguous patch emb., $T'$

Find the initial patch emb. $t = minCoordSum(T)$;
Set $T' = \{t\}$;
$T = T \setminus \{t\}$;
Initialize $prev_t = t$;
**while** $T \neq \emptyset$ **do**
  Set $closest = \infty$ ;
  **for** $t_i \in T$ **do**
    $dis = \| center(prev_t) - center(t_i) \|$;
    **if** $dis \leq closest$ **then**
      Set $closest = dis$;
      Set $index = i$;
    **end**
  **end**
  $T' = T' \cup \{t_index\}$;
  $T = T \setminus \{t_index\}$;
  $prev_t = t_index$;
**end**

### 3.2. Greedy Sequencer

Due to the previously observed benefits of having targets and context clustered together in close spatial proximity, a configuration known as a block in I-JEPA [2], we aim to sample patch embeddings that are spatially close to each other. As previously mentioned, point cloud data is permutation invariant to data feeding order, which implies that even if the indices of patch embeddings are sequential, they might not be spatially adjacent. Furthermore, our approach involves the selection of $M$ spatially contiguous blocks of encoded embeddings as the target while ensuring that the context does not include the patch embeddings corresponding to these embedding vectors (details in the next paragraph). To address these challenges, we apply a greedy sequencer that is applied after producing patch embeddings similar to z-ordering in PointGPT [7]. This sequencer orders patch embeddings based on their associated center points ( Algorithm 1). The process is initiated by selecting the center point with the lowest sum of coordinates ($minCoorSum(T)$) as the starting point, along with its associated patch embedding. In each subsequent step, the center point closest to the one previously chosen and its associated patch embedding are selected. This is iterated until the sequencer visits all of the center points. The resulting arrangement of patch embeddings ($T' = \{t_1^{'}, t_2^{'}, ..., t_r^{'}\}$) is in a sequence where contiguous elements are also spatially contiguous in most cases. This allows the shared computation of spatial proximity between context and target selection. At the same time, this also allows simpler implementation for context and target selection. It is worth noting, however, that selecting two adjacent patch embedding indices in this setting does not always guarantee spatial proximity; there might be a gap between them. While this is true, the experiment results show that this iterative ordering is effective enough in our JEPA architecture. Additionally, this rather simple approach is parallelized across batches, making it more efficient for large datasets or point clouds. Not only can we compute pairwise distances for all points within a batch in a single forward pass but also run the iterative process of simultaneously selecting the next closest point across the batch. This enables faster computation on modern GPUs, ensuring that the nearest points are selected efficiently while keeping the algorithm feasible even for large batch sizes.

### 3.3. JEPA Components

**Context and Target** Targets in Point-JEPA can be considered patch-level representations of the point cloud object, which the predictor aims to predict. As illustrated in Fig. 2, the target encoder initially encodes the patch embedding conventionally, and we randomly select $M$ possibly overlapping target blocks, which are sets of adjacent encoded embeddings. Specifically, we define $y(i) = \{y_j\}_{j \in B_i}$ as the $i^{\text{th}}$ target representation block, where $B_i$ denotes the set of mask indices for the $i^{\text{th}}$ block. Here, we denote the encoded embeddings as $y = \{y_1, y_2, \dots y_n\}$, where $y_k = f_{\bar{\theta}}(t_k^{'})$ is the representation associated with the $k^{th}$ centre point. It is important to note that masking for the target is applied to the embedding vectors derived from the patch embeddings that have passed through the transformer encoder $f_{\bar{\theta}}$. This ensures a high semantic level for the target representations.

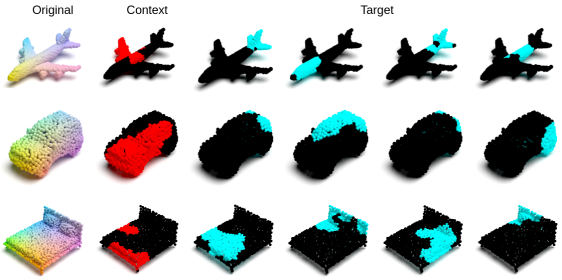



Figure 3. **Context and Targets.** We visualize the corresponding grouped points of context and target blocks. Here, we use (0.15, 0.2) for the target selection ratio and (0.4, 0.75) for the context selection ratio.

Context, on the other hand, is the representation of the point cloud object which is passed to the predictor to facilitate the reconstruction of target blocks. Unlike target blocks, masking is applied to the patch embeddings during the creation of context blocks. This allows the context-encoder $f_\theta$ to represent the uncertainties in the object's rep-

resentations when certain parts are masked. Specifically, we first select a subset of patch embeddings $\hat{T} \subseteq T'$ that are spatially contiguous. These selected embeddings are then fed to the context-encoder $f_\theta$ to generate a context block $x = \{x_j\}_{j \in B_x}$. To prevent trivial learning, we also ensure that the indices of patch embeddings chosen for the context differ from those for the targets. Furthermore, the patch embeddings $T'$ are sorted such that embeddings that are adjacent in the data feeding order are also spatially close. This organization simplifies the selection of contiguous target and context blocks, despite the aforementioned complexities of point cloud data representation.

**Predictor** The task of predictor $p_\phi$ given targets $y$ and context $x$ is analogous to the task of supervised prediction. Given a context as input $x$ along with a certain condition, it aims to predict the target representations $y$. Here, the condition involves the mask tokens, which are created from shared learned parameters, as well as positional encoding, created from centre points associated with the targets. That is

$$\hat{y}(i) = \{\hat{y}_j\}_{j \in B_i} = p_\phi\left(x, \{m_j\}_{j \in B_i}\right) \tag{1}$$

where $p_\phi(\cdot, \cdot)$ denotes the predictor and $\{m_j\}_{j \in B_i}$ denotes the mask token created from shared learnable parameter and positional encoding created from centre points.

**Loss** Because the predictor's task is to predict the representation produced by the target encoder, the loss can be defined to minimize the disagreement between the predictions and targets as follows.

$$\frac{1}{M}\sum_{i=1}^{M} D(\hat{y}(i), y(i)) = \frac{1}{M}\sum_{i=1}^{M}\sum_{j \in B_i} \mathcal{L}(\hat{y}_j, y_j) \tag{2}$$

Similar to Point2Vec [38], we utilize Smooth L1 loss to measure the dissimilarity between each corresponding element of the target and predicted embedding because of its ability to be less sensitive to the outliers.

**Parameter Update** We utilize AdamW [20] optimizer with cosine learning decay [19]. The target encoder and context encoder initially have identical parameters. The context encoder's parameters are updated via backpropagation, while the target encoders' parameters are updated using the exponential moving average of the context encoder parameters, that is $\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta$ where $\tau \in [0, 1]$ denotes the decay rate.

# 4. Experiments

In this section, we first describe the details of self-supervised pre-training. Further, we compare the performance of the learned representation to the state-of-the-art self-supervised learning methods in the point cloud domain that utilizes the ShapeNet [6] dataset in pre-training. We specifically evaluate the learned representation using linear probing, end-to-end fine-tuning, and a few-shot learning setting. Finally, ablation experiments are conducted to gain insights into the principal characteristics of Point-JEPA.

Table 1. **Linear Evaluation on ModelNet40 [36].** We compare Point-JEPA to self-supervised learning methods pre-trained on ShapeNet [6]. * signifies the linear evaluation results as indicated in [39, 40]. ** signifies results with Transformer backbone.

| methods | Overall Accuracy |
|---|---|
| Latent-GAN [1] | 85.7 |
| 3D-PointCapsNet [41] | 88.9 |
| STRL [16] | 90.3 |
| Sauder *et al.* [30] | 90.6 |
| Fu *et al.* [12] | 91.4 |
| Transformer-OcCo* [37] | 89.6 |
| Point-BERT* [37] | 87.4 |
| Point-MAE* [25] | 90.0 |
| Point-M2AE [39] | 92.9 |
| CluRender** [21] | 93.2 |
| Point-JEPA (Ours) | **93.7**$\pm$0.2 |

## 4.1. Self-Supervised Pre-training

We pre-train our model on training set of ShapeNet [6] following the previous studies utilizing the standard Transformer [34] architecture such as Point-MAE [25], Point-M2AE [39], PointGPT [7], and Point2Vec [38] for the fair comparison. The dataset consists of 41952 3D point cloud instances created from synthetic 3D meshes from 55 categories. The standard Transformer [34] architecture is used for the context and target encoder as well as the predictor. During pre-training, we set the number of center points to 64 and the group size to 32. The point tokenization is applied to the input point cloud containing 1024 points per object. We set the depth of the Transformer in the context and target encoder to 12 with the embedding width of 384 and 6 heads. For the predictor, we use the narrower dimension of 192 following I-JEPA [2]. The depth of the predictor is set to 6, and the number of heads is set to 6. Our experiments are conducted on NVIDIA RTX A5500 and NVIDIA A100 SXM4. We note that our method only takes 7.5 hours on RTX A5500 for pretraining (see Fig. 1) which is less than half of that of PointM2AE [39] and about 60% of Point2Vec [38] time requirement for pretraining. Adhering to the standard convention, we use overall accuracy for classification tasks and mean IoU for part segmentation tasks.

Table 2. **End-to-End Classification.** Overall accuracy on ModelNet40 [36] and ScanObjNN [32] with end-to-end fine-tuning. We specifically compare our methods to the method utilizing standard Transformer architecture pre-trained on ShapeNet [6] with only point cloud (no additional modality).

| Method | Reference | Overall Accuracy | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | **ModelNet40** | | | **ScanObjNN** | | | |
| | | #Points | +Voting | -Voting | #Points | OBJ-BG | OBJ-ONLY | OBJ-T50-RS |
| Point-BERT [37] | CVPR2022 | 1k | 93.2 | 92.7 | 1k | 87.4 | 88.1 | 83.1 |
| Point-MAE [25] | ECCV2022 | 1k | 93.8 | 93.2 | 2k | 90.0 | 88.3 | 85.2 |
| Point-M2AE [39] | NeurIPS2022 | 1k | 94.0 | 93.4 | 2k | 91.2 | 88.8 | 86.4 |
| Point2Vec [38] | GCPR2023 | 1k | **94.8** | **94.7** | 2k | 91.2 | 90.4 | 87.5 |
| PointGPT-S [7] | NeurIPS2023 | 1k | 94.0 | – | 2k | 91.6 | 90.0 | 86.9 |
| PointDiff [42] | CVPR2024 | – | – | – | 1k | **93.2** | **91.9** | **87.6** |
| Point-JEPA (Ours) | - | 1k | 94.1±0.1 | 93.8±0.2 | 2k | 92.9 ±0.4 | 90.1 ± 0.2 | 86.6 ± 0.3 |

## 4.2. Downstream Tasks

In this section, we report the performance of the learned representation on several downstream tasks. Following the previous studies [7, 37–39], we report the overall accuracy as a percentage. To account for variability across independent runs, we report the mean accuracy and standard deviation from 10 independent runs with different seeds, unless specified otherwise.

**Linear Probing.** After pre-training on ShapeNet [6], we evaluate the learned representation via linear probing on ModelNet40 [36]. Specifically, we freeze the learned context encoder and place the SVM classifier on top. To enforce invariance to geometric transformation, we utilize max and mean pooling on the output of the Transformer encoder [25, 38]. We utilize 1024 points for both training and test sets. As shown in Tab. 1, our method achieves state-of-the-art accuracy, providing $+0.8\%$ performance gain, showing the robustness of the learned representation.

**Few-Shot Learning** We conduct few-shot learning experiments on Modelnet40 [36] in $m$-way, $n$-shot setting as shown in Tab. 3. Specifically, we randomly sample $n$ instances of $m$ classes for training and select 20 instances of $m$ support classes for evaluation. For one setting, we run 10 independent runs under a fixed random seed on 10 different folds of dataset and report mean and standard deviation of overall accuracy. As shown in Tab. 3, our method exceeds the performance of current state-of-the-art in all settings and yields a $+1.1\%$ improvement in the most difficult 10-way 10-shot setting, showing the robustness of the learned representation of Point-JEPA, especially in the low-data regime.

**End-to-end Fine-Tuning** We also investigate the performance of the learned representation via end-to-end fine-tuning. After pre-training, we utilize the context encoder to extract the max and average pooled outputs. These outputs are then processed by a three-layer MLP for classification tasks. This class-specific head as well as the context encoder is fine-tuned end-to-end on ModelNet40 [36] and ScanObjectNN [32]. ModelNet40 consists of 12311 synthetic 3D objects from 40 distinct categories, while ScanObjectNN contains objects from 15 classes, each containing 2902 unique instances collected by scanning real-world objects. For ModelNet40, we sub-sample 1024 points per object and sample 64 center points with 32 points in each point patch. On the other hand, we utilize all 2048 points for the ScanObjNN dataset and sample 128 center points with 32 nearest neighbors for the grouped points. As shown in Tab. 2, our method achieves competitive results when compared to other state-of-the-art methods. Especially, in the OBJ-BG variant of the ScanObjNN [32] dataset, which

Table 3. **Result of Few-Shot classification on ModelNet40 [36].** 10 independent trials are completed under one setting. We report mean and standard deviation over 10 trials. ** signifies results with Transformer backbone.

| Method | Overall Accuracy | | | |
|---|---|---|---|---|
| | 5-way | | 10-way | |
| | 10-shot | 20-shot | 10-shot | 20-shot |
| Point-BERT [37] | 94.6±3.1 | 96.3±2.7 | 91.0±5.4 | 92.7±5.1 |
| Point-MAE [25] | 96.3±2.5 | 97.8±1.8 | 92.6±4.1 | 95.0±3.0 |
| Point-M2AE [39] | 96.8±1.8 | 98.3±1.4 | 92.3±4.5 | 95.0±3.0 |
| Point2Vec [38] | 97.0±2.8 | 98.7±1.2 | 93.9±4.1 | 95.8±3.1 |
| PointGPT-S [7] | 96.8±2.0 | 98.6±1.1 | 92.6±4.6 | 95.2±3.4 |
| CluRender** [21] | 97.2±2.3 | 98.4±1.3 | 93.7±4.0 | 96.0±2.9 |
| Point-JEPA (Ours) | **97.4**±2.2 | **99.2**±0.8 | **95.0**±3.6 | **96.4**±2.7 |

Table 4. **Masking Strategies.** Multi-block and single-block masking strategies and their effect on the learned representation.

| **Targets** | | | **Context** | | **OA** |
|---|---|---|---|---|---|
| Strategy | Ratio | Freq. | Strategy | Ratio | Modelnet40 Linear |
| random | (0.6, 0.6) | 1 | rest | – | 92.5 |
| contiguous | (0.6, 0.6) | 1 | rest | – | 92.3 |
| contiguous | (0.15, 0.2) | 4 | contiguous | (0.4, 0.75) | **93.7** |

Table 5. **Part Segmentation on ShapeNetPart [6].** $\text{mIoU}_C$ is the mean IoU for all part categories, and $\text{mIoU}_I$ is the mean IoU for all instances.

| Method | $\text{mIoU}_C$ | $\text{mIoU}_I$ |
|---|---|---|
| Transformer-OcCo [37] | 83.4 | 85.1 |
| Point-BERT [37] | 84.1 | 85.6 |
| Point-MAE [25] | 84.1 | 86.1 |
| Point-M2AE [39] | **84.9** | **86.5** |
| Point2Vec [38] | 84.6 | 86.3 |
| PointGPT-S [7] | 84.1 | 86.2 |
| Point-JEPA (Ours) | $83.9 \pm 0.1$ | $85.8 \pm 0.1$ |

presents a realistic representation of a point cloud that includes both the object and its background, our method achieves the overall accuracy marginally lower than Point-Diff [42] while an improvement of $+1\%$ over other SSL methods. This shows the learned representation obtained from pre-training with Point-JEPA can easily be transferred to a classification task.

**Part Segmentation** Following previous studies [7,25,37–39], we report the performance of Point-JEPA in part segmentation task. Here, we utilize the ShapeNetPart [6] dataset, consisting of 16881 objects from 16 categories. We utilize the identical architecture employed in Point2Vec [38] for this task. Specifically, we take the embeddings from $4^{\text{th}}$, $8^{\text{th}}$, and $12^{\text{th}}$ Transformer block and take the average of them. Then, we apply mean and average pooling to this averaged output. The max and mean pooled embedding along with a one-hot encoded class label of an object is used as a global feature vector for the object. The original averaged output is also up-sampled using the PointNet++ [27] feature propagation layer to create a feature vector for each point. Then, each feature vector is concatenated with the global feature vector. A shared MLP is utilized on this concatenated vector to predict the segmentation label for the given point. Although the Point-JEPA shows competitive results, as shown in Tab. 5, its performance is slightly worse than the state-of-the-art methods.

**Limitations** Point-JEPA's comparatively weaker performance in segmentation and its superior learned representation in classification indicate that the proposed approach emphasizes global features over local features. Additionally, the effectiveness of Point-JEPA for processing larger point clouds is uncertain due to redundancy in many areas in data and requires further study.

### 4.3. Ablation Study

We conducted thorough ablation studies to understand the effect of moving parts of Point-JEPA. We pre-train Point-JEPA on the ShapeNet [6] dataset under various settings and evaluate the learned representation with linear probing on the ModelNet40 [36] dataset.

**Masking Strategy.** We investigate the impact of the masking type on the performance. We consider single-block masking and multi-block masking. For single-block masking strategies, we consider random masking and contiguous masking. For random masking, we randomly select the 60% of indices out of all encoded embedding vectors. Similarly, for contiguous masking, embedding vectors that are spatially contiguous are selected. In this setting, all patch embeddings not corresponding to the selected target blocks are used as context (denoted as rest). On the other hand, in the multi-block masking setting, we sample multiple possibly overlapping spatially contiguous embedding vectors as targets, and we remove the corresponding patch embeddings already selected for targets during context selection. In this setting, we set the ratio range of 0.15 to 0.2 for targets, while we set the ratio range of 0.4 to 0.75 for context. As shown in Tab. 4, the single-block masking achieves sub-optimal performance regardless of the spatial contiguity of the target embedding. It shows that our method learns stronger representation by utilizing a smaller amount of targets with a larger frequency.

**Sequencer** At the heart of employing Point-JEPA for point cloud processing is the ability to translate point cloud data into a sequence of spatially contiguous patch embeddings. As shown in Tab. 6, we conduct the ablation experiment with different sequencer strategies including both the *z-order* [22] and *Hilbert-order* [31] space-filling curves,

Table 6. Experiment with different sequencers using the ModelNet40 linear evaluation and their corresponding training times on *RTX 5500 GPU* for 500 epochs.

| Sequencer | Modelnet40 Linear | Train Time |
|---|---|---|
| z-ordering [22] | 93.4 | 8.30h |
| Hilbert ordering [31] | 91.8 | 10.78h |
| Greedy (min index) | 92.7 | 7.47h |
| Greedy (min coordinate) | **93.7** | 7.47h |

as well as the proposed *greedy sequencer*. The *greedy sequencer* is evaluated in two versions: one starting with the point that has the minimum index in data feeding order, and the other starting with the point that has the minimum coordinate sum. The *greedy sequencer* with the minimum coordinate approach exhibits an overall accuracy improvement compared to all the other algorithms. Additionally, the *greedy sequencer* offers computational efficiency over *z-order* and *Hilbert-order* due to its ability to parallelize computations, as mentioned previously. Notably, the greedy sequencer with minimum coordinate sum initial point guarantees that the starting center point in the sequencer lies near the edge of the object. As shown in Tab. 6, selecting the point near the edge of the object as the starting point helps the model learn a stronger representation.

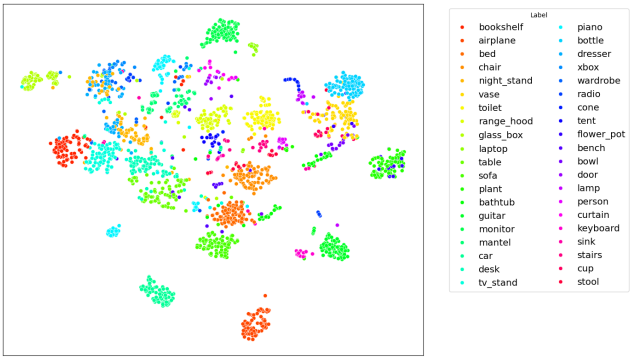



Figure 4. **Embedding Visualization on ModelNet40 [36].** We visualize the context encoder's learned representation with t-SNE [33].

**Number of Target Blocks.** We also consider the effect of the number of blocks chosen for targets on the performance of the learned representation while we keep the ratio for targets and context fixed. As shown in Tab. 7, the performance increases as we increase the number of targets. However, the performance decreases as you increase the number of target blocks after a specific frequency. We observe that our method benefits from having a sufficient amount of patch embeddings available for context encoding.

Table 7. **Number of Target blocks.** We change the number (frequency) of target blocks while keeping the other components fixed.

| **Targets** | | **Context** | **OA** |
|---|---|---|---|
| Ratio | Freq. | Ratio | Modelnet40 Linear |
| (0.15, 0.2) | 1 | (0.85, 1.0) | 93.0 |
| (0.15, 0.2) | 2 | (0.85, 1.0) | 93.5 |
| (0.15, 0.2) | 3 | (0.4, 0.75) | 93.4 |
| (0.15, 0.2) | 4 | (0.4, 0.75) | **93.7** |
| (0.15, 0.2) | 5 | (0.4, 0.75) | 93.4 |
| (0.15, 0.2) | 6 | (0.4, 0.75) | 93.2 |

### 4.4. Visualization

To qualitatively analyze the learned representation, we reduce the dimension of the learned representation by utilizing t-SNE [33]. We introduce max and mean pooling on the output of the context encoder, similar to the classification setup, and apply t-SNE on the pooled embedding. We visualize the learned representation on ModelNet40 [36] with no fine-tuning on the dataset. Despite being trained on the dataset, our context encoder produces discriminative features as shown in Fig. 4, showing the robustness of the learned representation.

## 5. Conclusion

This work introduced Point-JEPA, a joint embedding predictive architecture applied to point cloud objects. In order to efficiently select targets and context blocks even under the invariance property of point cloud data, we introduced a sequencer, which orders the center points and their corresponding patch embeddings by iteratively selecting the next closest center point. This eliminates the necessity of computing spatial proximity between every pair of patch embeddings or encoded embeddings when sampling the targets and context. Point-JEPA achieves state-of-the-art performance in downstream tasks, excelling in few-shot learning and linear evaluation. This makes Point-JEPA highly useful when there is a large amount of unlabeled data and a limited amount of labeled data. It is also worth noting that Point-JEPA converges much faster during pre-training, offering a more efficient pre-training alternative in the point cloud domain. Future work includes extending Point-JEPA for other downstream tasks such as object detection and scene level segmentation, and pre-training on large unlabeled hybrid datasets [7]. Additionally, the ability of JEPA to reduce dependency on large labeled datasets presents potential avenues for generating temporal predictive embeddings that anticipate point cloud evolution, thereby enhancing tasks like motion prediction, dynamic scene understanding, and anomaly detection.

## References


[1] Panos Achlioptas, Olga Diamanti, Ioannis Mitliagkas, and Leonidas J. Guibas. Representation learning and adversarial generation of 3d point clouds. *CoRR*, abs/1707.02392, 2017. 5

[2] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15619–15629, 2023. 1, 2, 3, 4, 5

[3] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mido Assran, and Nicolas Ballas. V-jepa: Latent video prediction for visual representation learning. 2023. 1

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *CoRR*, abs/2005.14165, 2020. 2

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. *CoRR*, abs/2104.14294, 2021. 2

[6] Angel X. Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, Jianxiong Xiao, Li Yi, and Fisher Yu. ShapeNet: An Information-Rich 3D Model Repository. Technical Report arXiv:1512.03012 [cs.GR], Stanford University — Princeton University — Toyota Technological Institute at Chicago, 2015. 5, 6, 7

[7] Guangyan Chen, Meiling Wang, Yi Yang, Kai Yu, Li Yuan, and Yufeng Yue. Pointgpt: Auto-regressively generative pre-training from point clouds. *Advances in Neural Information Processing Systems*, 36, 2024. 2, 3, 4, 5, 6, 7, 8

[8] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 1597–1607. PMLR, 13–18 Jul 2020. 2

[9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. 2

[10] Bi'an Du, Xiang Gao, Wei Hu, and Xin Li. Self-contrastive learning with hard negative sampling for self-supervised point cloud learning. In *Proceedings of the 29th ACM International Conference on Multimedia*, pages 3133–3142, 2021. 2

[11] Y. Eldar, M. Lindenbaum, M. Porat, and Y.Y. Zeevi. The farthest point strategy for progressive image sampling. In *Proceedings of the 12th IAPR International Conference on Pattern Recognition, Vol. 2 - Conference B: Computer Vision Image Processing. (Cat. No.94CH3440-5)*, pages 93–97 vol.3, 1994. 3

[12] Kexue Fu, Peng Gao, Renrui Zhang, Hongsheng Li, Yu Qiao, and Manning Wang. Distillation with contrast is all you need for self-supervised point cloud representation learning, 2022. 5

[13] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Ávila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. *CoRR*, abs/2006.07733, 2020. 2

[14] Yulan Guo, Hanyun Wang, Qingyong Hu, Hao Liu, Li Liu, and Mohammed Bennamoun. Deep learning for 3d point clouds: A survey. *IEEE transactions on pattern analysis and machine intelligence*, 43(12):4338–4364, 2020. 1

[15] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. Masked autoencoders are scalable vision learners. *CoRR*, abs/2111.06377, 2021. 2

[16] Siyuan Huang, Yichen Xie, Song-Chun Zhu, and Yixin Zhu. Spatio-temporal self-supervised representation learning for 3d point clouds, 2021. 5

[17] Ashish Jaiswal, Ashwin Ramesh Babu, Mohammad Zaki Zadeh, Debapriya Banerjee, and Fillia Makedon. A survey on contrastive self-supervised learning. *Technologies*, 9(1):2, 2020. 1, 2

[18] Yann LeCun. A path towards autonomous machine intelligence, 2022. 1, 3

[19] Ilya Loshchilov and Frank Hutter. SGDR: stochastic gradient descent with restarts. *CoRR*, abs/1608.03983, 2016. 5

[20] Ilya Loshchilov and Frank Hutter. Fixing weight decay regularization in adam. *CoRR*, abs/1711.05101, 2017. 5

[21] Guofeng Mei, Cristiano Saltori, Elisa Ricci, Nicu Sebe, Qiang Wu, Jian Zhang, and Fabio Poiesi. Unsupervised point cloud representation learning by clustering and neural rendering. *International Journal of Computer Vision*, pages 1–19, 2024. 5, 6

[22] M Morton, G. *A Computer Oriented Geodetic Data Base and a New Technique in File Sequencing.* International Business Machines, 1966. 7, 8

[23] Norman Mu, Alexander Kirillov, David Wagner, and Saining Xie. Slip: Self-supervision meets language-image pre-training. In *European conference on computer vision*, pages 529–544. Springer, 2022. 2

[24] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 2

[25] Yatian Pang, Wenxiao Wang, Francis EH Tay, Wei Liu, Yonghong Tian, and Li Yuan. Masked autoencoders for point cloud self-supervised learning. In *European conference on computer vision*, pages 604–621. Springer, 2022. 2, 3, 5, 6, 7

[26] Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. Pointnet: Deep learning on point sets for 3d classification and segmentation, 2017. 3

[27] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. Pointnet++: Deep hierarchical feature learning on point sets in a metric space. *CoRR*, abs/1706.02413, 2017. 7

[28] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. 2

[29] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *CoRR*, abs/1910.10683, 2019. 2

[30] Jonathan Sauder and Bjarne Sievers. Context prediction for unsupervised deep learning on point clouds. *CoRR*, abs/1901.08396, 2019. 5

[31] John Skilling. Programming the hilbert curve. volume 707, pages 381–387, 04 2004. 7, 8

[32] Mikaela Angelina Uy, Quang-Hieu Pham, Binh-Son Hua, Duc Thanh Nguyen, and Sai-Kit Yeung. Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data. *CoRR*, abs/1908.04616, 2019. 6

[33] Laurens van der Maaten and Geoffrey E. Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008. 8

[34] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017. 3, 5

[35] Yi Wang, Conrad M Albrecht, Nassim Ait Ali Braham, Lichao Mou, and Xiao Xiang Zhu. Self-supervised learning in remote sensing: A review. *IEEE Geoscience and Remote Sensing Magazine*, 10(4):213–247, 2022. 2

[36] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1912–1920, 2015. 1, 5, 6, 7, 8

[37] Xumin Yu, Lulu Tang, Yongming Rao, Tiejun Huang, Jie Zhou, and Jiwen Lu. Point-bert: Pre-training 3d point cloud transformers with masked point modeling. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 19313–19322, 2022. 2, 5, 6, 7

[38] Karim Abou Zeid, Jonas Schult, Alexander Hermans, and Bastian Leibe. Point2vec for self-supervised representation learning on point clouds. In *DAGM German Conference on Pattern Recognition*, pages 131–146. Springer, 2023. 2, 3, 5, 6, 7

[39] Renrui Zhang, Ziyu Guo, Rongyao Fang, Bin Zhao, Dong Wang, Yu Qiao, Hongsheng Li, and Peng Gao. Point-m2ae: Multi-scale masked autoencoders for hierarchical point cloud pre-training, 2022. 5, 6, 7

[40] Renrui Zhang, Liuhui Wang, Yu Qiao, Peng Gao, and Hongsheng Li. Learning 3d representations from 2d pre-trained models via image-to-point masked autoencoders, 2022. 5

[41] Yongheng Zhao, Tolga Birdal, Haowen Deng, and Federico Tombari. 3d point-capsule networks. *CoRR*, abs/1812.10775, 2018. 5

[42] Xiao Zheng, Xiaoshui Huang, Guofeng Mei, Yuenan Hou, Zhaoyang Lyu, Bo Dai, Wanli Ouyang, and Yongshun Gong. Point cloud pre-training with diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22935–22945, 2024. 6, 7